A.智乃的二进制

~~不会~~

B.智乃的瓷砖

根据 $i+j$ 的奇偶性判断就行。

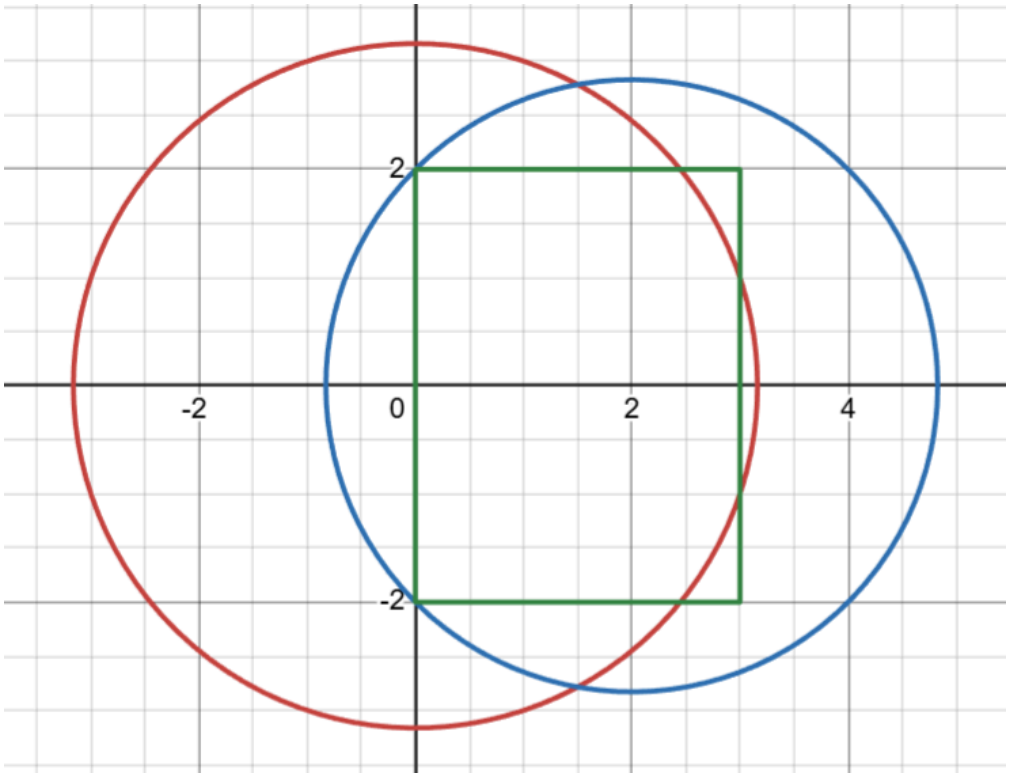
```
void solve(){
    ll n = read(), m = read();
    for(ll i=0;i<n;i++){
        for(ll j=0;j<m;j++){
            putchar((i+j+1)&1?'/':'\\');
        }
        putchar('\n');
    }

}
```

C.智乃的草坪

由题意发现整个区域关于 $y=0$ 对称，圆也关于 $y=0$ 对称。

画一下图可以发现， $L1: y=r, x\in[0,c]$ 这条线段被覆盖了和整个区域被覆盖是充要条件。



因此我们需要找到的时间需要满足只用 k 个圆，能覆盖满 $L1$ 。

而随着时间增加，能用的圆单增，覆盖的面积也单增，因此可以二分。

二分时，找到圆和 $L1$ 的相交弦，判断用最少的线段覆盖的数量是否小于 k。

时间复杂度 $O(Knlogn)$， K 为二分次数。

注意浮点数判等的 eps。

```
void solve(){
    ll n, k, r, c;cin>>n>>k>>r>>c;
    vector<PLL> memo(n);
    for(ll i=0;i<n;i++) cin>>memo[i].fi>>memo[i].se;
    function<ll(db)> check = [&](db mid){
        vector<pair<db, db>> segs;
        for(auto [p, v]:memo){
            db curr = v * mid;
            if(curr>r||fabs(curr-r)<1e-6){
                db d = sqrt(curr*curr-r*r);
                segs.pb({max((db)0.0, db(p)-d), min((db)c, p+d)});
            }
        }
        sort(all(segs));
        ll cnt=0, idx=0;
        db end=0;
        while(end < c){
            db mx = -INF;
            ll f = 0;
            while(idx<segs.size()&&(segs[idx].fi<end||fabs(segs[idx].fi-end)<1e-
6)){
                f = 1;
                mx = max(mx, segs[idx].se);
                idx ++;
            }
            if(!f||mx<end||fabs(mx-end)<1e-6){
                return 0ll;
            }
            cnt++;
            end=mx;
        }
        return (cnt <= k)?1ll:0ll;
    };
    db L = 0, R = 1e7;
    for(ll i=0;i<100;i++){
        db mid = (L+R)/2;
        if(check(mid)){
            R = mid;
        }
        else{
            L = mid;
        }
    }
    cout<<fixed<<setprecision(6)<<(L+R)/2<<'\n';
}
```

D.智乃的果子

优先队列模拟一下合并过程即可。

从小到大取的过程中，对于同一堆果子，优先和自己合并，否则和次大合并。

时间复杂度 $O(nlogn)$。

注意不能用 `pq.size() == 1` 判断是否结束，因为此时可能有多个重复的。

```
void solve(){
    ll n = read();
    map<ll, ll> memo;
    for(ll i=0;i<n;i++){
        ll c = read(), w = read();
        memo[w] += c;
    }
    priority_queue<PLL, vector<PLL>, greater<PLL>> pq;
    ll tot = 0;
    for(auto [k, v]:memo){
        pq.push({k, v});
        tot += v;
    }
    ll ans = 0;
    while(tot>1){
        auto [w, c] = pq.top();
        pq.pop();
        ll t = c/2;
        ll r = c%2;
        tot-=t;
        ans = (ans + t * (2 * w)%MOD)%MOD;
        if(t > 0) pq.push({2*w, t});
        if(r){
            if(pq.size()){
                auto [w2, c2] = pq.top();
                pq.pop();
                c2 -= 1;
                ans = (ans + w + w2)%MOD;
                pq.push({w+w2, 1});
                tot -= 1;
                if(c2>0){
                    pq.push({w2, c2});
                }
            }
            else{
                pq.push({r, 1});
            }
        }
    }
    print(ans);
}
```

E.智乃的最大子段和取模

不难想到前缀和+枚举每个 i。

对于 i，为了最大化最大子段和，我们可以

- 选择前面最小的前缀和
- 选择最小的比其大的前缀和

例如 $pre_i = 4, p = 6$ ，已有前缀和 $0\ \ 5$ 时，可以选择 $4 - 0 = 4$ 或者 $(4 - 5 + 4)\%p = 5$。

时间复杂度 $O(nlogn)$。

```
void solve(){
    ll n = read(), p = read();
    vector<ll> a(n+1), pre(n+1);
    for(ll i=1;i<=n;i++) a[i] = read(), pre[i] = (pre[i-1] + a[i])%p;
    ll mx = -1, ansl = -1, ansr = -1;
    map<ll, ll> memo;
    set<ll> pres;
    pres.insert(0);
    memo[0] = 0;
    for(ll i=1;i<=n;i++){
        if(*pres.begin() <= pre[i]){
            ll res = pre[i] - (*pres.begin());
            if(res > mx){
                mx = res;
                ansl = memo[(*pres.begin())] + 1;
                ansr = i;
            }
        }
        auto it = pres.upper_bound(pre[i]);
        if(it != pres.end()){
            ll res = (pre[i] - (*it) + p)%p;
            if(res > mx){
                mx = res;
                ansl = memo[(*it)] + 1;
                ansr = i;
            }
        }
        pres.insert(pre[i]);
        memo[pre[i]] = i;
    }
    pt(ansl-1), putchar(' '), pt(ansr-1), putchar(' '), pt(mx), putchar('\n');
}
```

F.智乃的最大子段和取模

~~如果是两个任意字符串，这是AC自动机fail树上dp+矩阵快速幂。~~

不难发现，两个字符串只能在末尾重叠成 `qcjjkktd` 。

设我们使用了 x 个 `qcjjkkt`， y 个 `td`。

显然我们重叠越多越节省长度，即重叠 $min(x, y)$ 个。

则需要满足 $7x + 2y - min(x, y) \le n$。

即

$$\begin{cases} 7x+y\le n, & x\ge y \\ 6x+2y\le n, & x<y \end{cases}$$

要最大化 $ax+by$，手玩一下可以发现基本取在 $x=0$ 、 $y=0, x=y$ 附近，遍历一下即可。

时间复杂度 $O(1)$。

```
void solve(){
    ll n = read(), a = read(), b = read();
    ll ans = 0;
    for(auto xx:vector<ll>{0, n/7, n/8, n/6}){
        for(ll d=-3;d<=3;d++){
            ll x = xx + d;
            if(x < 0) continue;
            if(7*x <= n){
                ll y = min(n-7*x, x);
                ans = max(ans, a*x+b*y);
            }
            if(6*x <= n){
                ll y = (n-6*x)/2;
                if(y >= x) ans = max(ans, a*x+b*y);
            }
        }

    }
    print(ans);
}
```

G.智乃的箭头魔术

对着图手玩一下即可。

时间复杂福 $O(n)$。

答案为

```
3132333010010310230010130130330130312312210210010321300120122322322101123223211001
0030130300312103 32
```

```
void solve(){
    string s =
"0112233445142015320125410214530214510214102302142025101203201451451522302514203214510021454101002532";
    ll cur = 0;
    for(auto v:s){
        if(v == '0'){
            cur = 3 - cur;
        }
        else if(v == '1'){
            if(cur == 1 || cur == 3) cur = 4 - cur;
        }
        else if(v == '2'){
            if(cur  == 0 || cur == 1) cur = 1 - cur;
            else cur = 5 - cur;
        }
```

```
        else if(v == '3'){
            if(cur == 0 || cur == 2) cur = 2 - cur;
        }
        else if(v == '4'){
            cur = (cur + 1) % 4;
        }
        else{
            cur = (cur + 3) % 4;
        }
        pt(cur);
    }
}
```

H.智乃的矩阵

不难想到寻找操作中的不变量。

显然所有格子之和必须是 n^2 的倍数。

把矩阵画上黑白格，则黑格之和不变，白格之和不变。

对于每一行/列，操作过程中奇偶性不变。

判断这几个不变量是否成立就能过了。

时间复杂度 $O(n^2)$。

```
void solve(){
    ll n = read();
    vector<vector<ll>> g(n+1, vector<ll>(n+1));
    ll sum = 0;
    vector<ll> rs(n+1), cs(n+1);
    vector<ll> t(2);
    for(ll i=1;i<=n;i++){
        for(ll j=1;j<=n;j++){
            g[i][j] = read();
            t[(i+j)&1] += g[i][j];
            rs[i] += g[i][j];
            cs[j] += g[i][j];
            sum += g[i][j];
        }
    }
    if(sum % (n*n)){
        puts("No");
        return;
    }
    ll tar = sum / (n*n);
    if(t[0] != tar * ((n*n+1)/2)){
        puts("No");
        return;
    }
    if(t[1] != tar * ((n*n)/2)){
        puts("No");
        return;
    }
    for(ll i=1;i<=n;i++){
```

```
        if((rs[i]%2)^(n*tar%2)){
            puts("No");
            return;
        }
        if((cs[i]%2)^(n*tar%2)){
            puts("No");
            return;
        }
    }
    puts("Yes");
}
```

I.智乃挖坑

对于 $\max(f_i - |j - p_i|, 0)$ ，可以化成

$$\begin{cases} j + (f_i - p_i) \le n, & j \le p_i \\ -j + (f_i + p_i) \le n, & j > p_i \end{cases}$$

我们可以用两个差分数组，一个记录 j 前面的系数，另一个记录常数项。

复原时 i 处挖的深度即为 $a1_i * i + a2_i$。

再二分答案即可。

时间复杂度 $O(nlogm)$。

```
void solve(){
    ll n = read(),  m = read(), h = read();
    vector<PLL> ops(m+1);
    for(ll i=1;i<=m;i++){
        ops[i] = {read(), read()};
    }
    function<ll(ll)> check = [&](ll mid){
        vector<ll> d1(n+2), d2(n+2);
        for(ll i=1;i<=mid;i++){
            ll l1 = max(1ll,ops[i].fi-ops[i].se+1);
            ll r1 = ops[i].fi;
            if(l1 <= r1){
                d1[l1] += 1;
                d1[r1+1] -= 1;
                d2[l1] += (ops[i].se-ops[i].fi);
                d2[r1+1] -= (ops[i].se-ops[i].fi);
            }
            ll l2 = ops[i].fi+1;
            ll r2 = min(n,ops[i].fi+ops[i].se-1);
            if(l2 <= r2){
                d1[l2]-=1;
                d1[r2+1]+=1;
                d2[l2]+=(ops[i].se+ops[i].fi);
                d2[r2+1]-=(ops[i].se+ops[i].fi);
            }

        }
        for(ll i=1;i<=n;i++){
            d1[i] += d1[i-1];
```

```
            d2[i] += d2[i-1];
            if(d1[i]*i+d2[i]>h) return 1ll;
        }
        return 0ll;
    };
    ll l = 1, r = m, ans = -1;
    while(l <= r){
        ll mid = (l+r)>>1;
        if(check(mid)){
            r = mid - 1;
            ans = mid;
        }
        else{
            l = mid + 1;
        }
    }
    if(ans == -1){
        puts("No");
    }
    else{
        puts("Yes");
        print(ans);
    }
}
```

J.智乃的幻方

根据题意判断即可。

经典结论是三阶幻方每行每列对角线之和一定是 15 。

```
void solve(){
    vector<vector<ll>> g(3, vector<ll>(3));
    map<ll, ll>  memo;
    for(ll i=0;i<=2;i++)for(ll j=0;j<=2;j++) g[i][j] = read(), memo[g[i][j]]++;
    ll f = 1;
    for(ll i=0;i<=2;i++){
        if(g[i][0]+g[i][1]+g[i][2]!=15) f = 0;
        if(g[0][i]+g[1][i]+g[2][i]!=15) f = 0;
    }
    if(g[0][0]+g[1][1]+g[2][2]!=15) f = 0;
    if(g[2][0]+g[1][1]+g[0][2]!=15) f = 0;
    for(ll i=1;i<=9;i++){
        if(!memo[i]) f = 0;
    }
    puts(f?"Yes":"No");
}
```

C++ 火车头

```
// FZANOTFOUND
```

```
#include <bits/stdc++.h>
using namespace std;

#define pb push_back
#define eb emplace_back
#define fi first
#define se second
#define ne " -> "
#define sep "======"
#define fastio ios::sync_with_stdio(false);cin.tie(0);
#define all(a) a.begin(), a.end()

typedef long long ll;
typedef unsigned long long ull;
typedef long double db;
typedef pair<long long,long long> PLL;
typedef tuple<ll,ll,ll> TLLL;
const ll INF = (ll)2e18+9;
const ll MOD = 1000000007;
//const ll MOD = 998244353;
const db PI = 3.14159265358979323;

//io functions
inline void rd(ll &x){x=0;short f=1;char c=getchar();while((c<'0'||c>'9')&&c!='-
') c=getchar();if(c=='-') f=-1,c=getchar();while(c>='0'&&c<='9') x=x*10+c-
'0',c=getchar();x*=f;}
inline ll read(){ll x=0;short f=1;char c=getchar();while((c<'0'||c>'9')&&c!='-')
c=getchar();if(c=='-') f=-1,c=getchar();while(c>='0'&&c<='9') x=x*10+c-
'0',c=getchar();x*=f;return x;}
inline void pt(ll x){if(x<0) putchar('-'),x=-x;if(x>9)
pt(x/10);putchar(x%10+'0');}
inline void print(ll x){pt(x), puts("");}
inline void print(PLL x){pt(x.fi), putchar(' '), pt(x.se), putchar('\n');}
inline void print(vector<ll> &vec){for(const auto t:vec)pt(t),putchar('
');puts("");}
inline void print(const map<ll, ll>& g) {for(const auto& [key, value]:g)
{cout<<"key: "<<key<<ne<<value<<" ";}puts("");}
inline void print(vector<PLL> &vec){puts(sep);for(const auto v:vec)
{print(v);}puts(sep);}
inline void print(const map<ll, vector<ll>>& g) {for (const auto& [key, value] :
g) { cout << "key: " << key << ne;for (const auto& v : value) {cout << v << "
";}cout << endl;}}

//fast pow
ll ksm(ll a, ll b=MOD-2, ll M=MOD){a%=M;ll res=1;while(b){if(b&1){res=
(res*a)%M;}a=(a*a)%M;b>>=1;}return res;}

mt19937_64 rng(chrono::steady_clock::now().time_since_epoch().count());//rng()
ull randint(ull l, ull r){uniform_int_distribution<unsigned long long> dist(l,
r);return dist(rng);}

void init(){

}

void solve(){
```

```
}


int main(){
    init();
    ll t = 1;
    //t = read();
    while(t--){
        solve();
    }
}
```